# Midterm Progress Presentation Script

CIS 4930 ML for Computer Systems, Team 01

Do Learned Indexes Still Win on Modern Hardware?

Team: Riyon Praveen, Vishakha Tiwari, Sevara Badriddinova, Tuan Khang Phan. Presenter: Tuan Khang Phan. Target duration: about 4.5 minutes, followed by questions.

## 1. Project Overview

Database systems rely on index structures to locate records quickly. The most widely used structure, the B+tree, finds a key through a sequence of comparisons. Each comparison may require a slow access to main memory.

Kraska et al. (2018) proposed learned indexes, which replace the tree with a model that predicts the position of a key in a sorted array. They reported lookups up to 70 percent faster than a cache-optimized B+tree, with an order of magnitude less memory. Those results were obtained on hardware that is now several years old.

Our project determines whether, and under which conditions, this advantage still holds on current processors. We compare four learned indexes (RMI, PGM-index, RadixSpline and ALEX) against a B+tree, a hash table and binary search, using the SOSD benchmark on real and synthetic datasets. We measure latency, throughput and memory, use hardware performance counters to explain the results, test known weaknesses such as skewed data and inserts, and repeat key experiments on AMD, Intel and ARM processors.

At the midterm, a first experiment on one processor shows a 6.5 times speedup over binary search on uniform data, and a much smaller gain on skewed data.

## 2. Presentation Structure

| Part | Time |
|---|---|
| Box 1. The decision | 0:30 |
| Box 2. Beat what, exactly? | 0:25 |
| Box 3. Our approach | 0:35 |
| Box 4. First result | 1:20 |
| Box 5. What surprised us | 1:00 |
| Box 6. Next six weeks | 0:30 |
| Gate check and closing | 0:20 |

Text in square brackets is a presenter action and is not spoken.

## 3. Script

**Box 1. The decision (0:30)**

- Hello, everyone. We are Team 1, and our project examines whether learned indexes still outperform traditional index structures on modern hardware.
- Every database lookup must determine where a key is stored within a sorted array. Today, this decision is typically made by a B+tree, which moves from the root to a leaf through a sequence of comparisons. An imprecise step never produces an incorrect result, but each additional step can cause a cache miss, which costs roughly one hundred nanoseconds.

**Box 2. Beat what, exactly? (0:25)**

- Our baseline is binary search over the sorted array, which is the reference baseline in the SOSD benchmark. On our test machine, it requires 525 nanoseconds per lookup at one hundred million keys. A tuned B+tree will be added as a stronger baseline in the next phase.

**Box 3. Our approach (0:35)**

- A learned index replaces the tree with a model that predicts the position of a key. We will evaluate four designs: the Recursive Model Index, or RMI, the PGM-index, RadixSpline, and ALEX, which also supports inserts.
- Training labels are available at no cost, since they are simply the positions of the keys. Each model also records its maximum prediction error, so a lookup requires one prediction followed by a search within a small, bounded window. The result is therefore always exact.

**Box 4. First result (1:20)**

- For our first result, we implemented a two-stage RMI and compared it with binary search on uniform and lognormal key distributions, ranging from one hundred thousand to one hundred million keys.
- *[Point to the axes.]* The horizontal axis shows the number of keys, and the vertical axis shows the median time per lookup. The dashed lines represent binary search, and the solid lines represent the learned index.
- *[Point to the dotted line.]* Beyond this line, the data no longer fits in the 16-megabyte L3 cache. Binary search must then wait on main memory several times per lookup, and its latency rises sharply.
- *[Point to the solid black line.]* The learned index remains nearly flat. At one hundred million keys, it reduces the lookup time from 525 to 81 nanoseconds. This is a speedup of approximately 6.5 times, with a model of only 2.3 megabytes.
- These measurements were taken on an AMD Ryzen AI 7 350 laptop processor. Each point is the median of five runs of two million random lookups, following a warm-up run.

**Box 5. What surprised us (1:00)**

- The most significant finding concerns the key distribution. With the same model size, lookups on lognormal keys required 226 nanoseconds, compared with 81 nanoseconds on uniform keys. With a very small model, the learned index was even slower than binary search.
- A simple linear model cannot capture a skewed distribution, so its error window, and therefore its search cost, grows. This indicates that results on uniform data overstate the benefit of learned indexes.
- We also observed that this processor combines two types of cores. Without pinning the benchmark to a single core, the results were approximately ten percent slower and considerably less stable.

**Box 6. Next six weeks (0:30)**

- Over the next six weeks, the decisive experiment is the full SOSD benchmark on four real-world datasets, with hardware performance counters, on AMD, Intel, and ARM processors. The principal risk is that the Linux perf tool cannot access hardware counters under WSL or macOS. Our fallback is to use bare-metal cloud instances on AWS.

**Gate check and closing (0:20)**

- Regarding the gate check, an incorrect prediction is safe, the labels are free, and the model is small. Whether the approach beats a tuned B+tree, and whether it withstands insert-heavy workloads, remains to be determined.
- We would like to close with a question for the class: should our next priority be additional real-world datasets, or insert-heavy workloads? Thank you.

## 4. Key Figures

| Figure | Meaning |
|---|---|
| 525 ns | Binary search, 100 million keys |
| 81 ns | Learned index, 100 million uniform keys |
| 6.5x | Speedup over binary search (525 / 81) |
| 2.3 MB | Size of the learned model |
| 226 ns | Learned index on lognormal keys, same model size |
| 16 MB | L3 cache size (dotted line on the plot) |

## 5. Anticipated Questions

**Why compare against binary search rather than a B+tree?**

- Binary search is the reference baseline in SOSD. A tuned B+tree is usually faster on large data, so the 6.5 times speedup should be read as an upper bound. The B+tree is the next baseline we will add.

**The learned index was tuned, but the baseline was not. Is the comparison fair?**

- Binary search has no parameter to tune. Even our smallest model, at 0.6 megabytes, requires 97 nanoseconds at one hundred million keys, which is still about five times faster.

**Why does binary search slow down beyond 16 megabytes?**

- One hundred million keys require about 27 search steps. The first steps remain in cache, but the final steps each wait on main memory, at roughly 100 nanoseconds per access.

**How do learned indexes handle inserts?**

- A static model must be retrained after inserts. ALEX reserves gaps in its arrays so that it can insert in place. Insert-heavy workloads are part of our next phase.

**Do the original conclusions still hold?**

- On this machine, for read-only uniform data against binary search, they do. Whether they hold on real data, against a tuned B+tree, and on other processors is the subject of the remaining work.

## 6. Delivery Notes

- Speak at a measured pace, approximately 120 words per minute, and pause briefly after each key figure.
- During Box 4, stand beside the screen and point to each line as it is described.
- If a question concerns work not yet completed, state this directly and describe how the plan addresses it.